The Battleship Approach to the Low Resource Entity Matching Problem

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Worcester Polytechnic Institute, USA

Entity matching, a core data integration problem, is the task of deciding whether two data tuples refer to the same real-world entity. Recent advances in deep learning methods, using pre-trained language models, were proposed for resolving entity matching. Although demonstrating unprecedented results, these solutions suffer from a major drawback as they require large amounts of labeled data for training, and, as such, are inadequate to be applied to *low resource entity matching* problems. To overcome the challenge of obtaining sufficient labeled data we offer a new *active learning* approach, focusing on a selection mechanism that exploits unique properties of entity matching. We argue that a distributed representation of a tuple pair indicates its informativeness when considered among other pairs. This is used consequently in our approach that iteratively utilizes space-aware considerations. Bringing it all together, we treat the low resource entity matching problem as a Battleship game, hunting indicative samples, focusing on positive ones, through awareness of the latent space along with careful planning of next sampling iterations. An extensive experimental analysis shows that the proposed algorithm outperforms state-of-the-art active learning solutions to low resource entity matching, and although using less samples, can be as successful as state-of-the-art fully trained known algorithms.

CCS Concepts: • **Information systems** → **Entity resolution**; • **Computing methodologies** → **Active learning settings**.

Additional Key Words and Phrases: entity resolution, entity matching, active learning

ACM Reference Format:
Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4 (SIGMOD), Article 224 (December 2023), 25 pages. https://doi.org/10.1145/3626711

1 INTRODUCTION

Entity matching, with variations as entity resolution, record linkage, record deduplication and more, is a core data integration task that plays a crucial role in any data project life cycle [8, 15]. Essentially, entity matching aims to identify dataset tuples that refer to the same real-world entities.

Traditional methods for entity matching and its variations focused on string similarity [20, 21, 27, 31] and probabilistic approaches [10], followed by rule-based methods [58, 59]. Learning-based approaches were also suggested [4, 26] with an increased focus on deep learning in recent years [13, 14, 24, 25, 28, 41, 68], culminating in the use of pre-trained language models, and specifically BERT [9], fine-tuned for entity matching [7, 29, 30, 45].

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il, sbargen@campus.technion.ac.il; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il; Roee Shraga, Worcester Polytechnic Institute, Worcester, Massachusetts, USA, rshraga@wpi.edu.

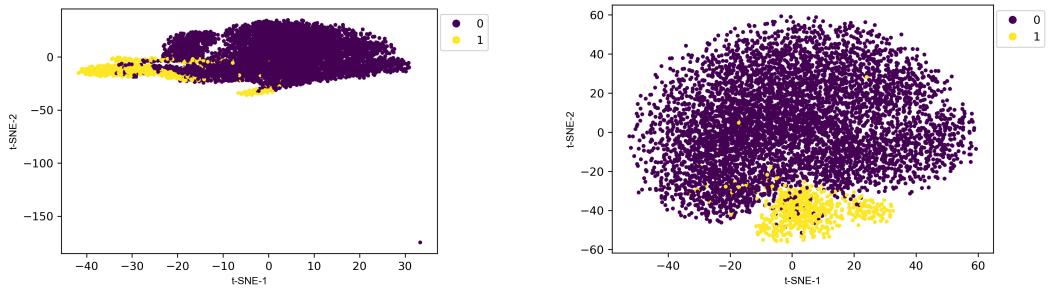
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/12-ART224 $15.00
https://doi.org/10.1145/3626711

Machine learning approaches to entity matching are by-and-large supervised, requiring labeled data for training, and in the case of deep learning, massive amounts of them [34, 46, 60]. Obtaining labeled data is challenging, mostly due to the need for intensive human expert labor that accompanies the labeling process [33, 34, 55]. This, in turn, calls for methods that reduce the labeling process load while maintaining an accurate trained matcher.

Several approaches were suggested over the years to overcome the dependence on large amounts of data, among which, unsupervised [3, 65, 66] and active learning methods [47, 53]. Active learning, the most prevalent approach to the insufficient labeled data problem, offers methods for training using a limited amount of labeled data. This is done by zeroing in on informative samples to be labeled and used for training. An active learning framework uses selection criteria (*e.g.*, variations of uncertainty [25, 38, 48] and centrality [67, 69]) to pinpoint the samples that are expected to improve the model performance the most. These selected samples are then sent to a labeling oracle and added to the available training set in following iterations.

A key tool which we use to quantify both uncertainty and centrality of a sample (a record pair in our case) is its vector-space latent representation. Traditional entity matching approaches constructed pair feature vector using widely used similarity measures [4, 26, 65] or learnable attribute-wise similarity vector using neural networks [24, 41]. Contemporary solutions [7, 29, 45], which demonstrate superior performance and do not require feature engineering, yield an embedding vector as alternative to feature vectors when representing the input (pair) with respect to the task.

In this work we offer a solution to low resource entity matching tasks, where there is a limited budget for labeling. In particular, we attend to the labeled data scarcity problem using active learning, proposing a novel strategy tailored to the specific characteristics of entity matching. Entity matching usually suffers from inherent label imbalance, where the number of negative (non-match) pairs overshadows the number of positive (match) ones [2]. When training with insufficient amount of data, this property is often problematic since the model may struggle to generalize to unseen minority class samples (in this case, matching pairs). To tackle this challenge, we ensure a balanced sampling of positive and negative samples using high-dimensional latent space encoding for tuple pairs. Specifically, we argue that vector-space similarity between pair representations reflects, with high probability, agreement of their labels. We next use this notion to carefully guarantee a balanced sampling, although positive pairs are harder to be found.



(a) Amazon-Google (b) Walmart-Amazon

Fig. 1. Visualization of pair distribution by t-SNE, partitioned into match and non-match pairs. Using representative vectors (with dimension of 768) of a fully trained models we demonstrate that match pairs tend to gather together.

Example 1. *Figure 1 provides an illustration of the latent space of tuple pairs over well known benchmarks using t-SNE [64]. Figure 1a presents the widely used Amazon-Google dataset,*[1] *which consists of 11,460 pairs, with only 1,167 (∼10%) labeled as match samples. To demonstrate the pairs behavior, we trained a DITTO [29] model with the fully available train set (60% of the entire dataset). After training, we extracted the latent representations (dimension of 768) generated for each tuple pair and applied t-SNE to create a human-readable 2-dimensional reduced space, where the yellow dots represent match pairs and the purple dots represent non-match pairs. As illustrated, there is a concentration of match pairs in a few main areas of the latent space, forming a cohesive and clear partition between match and non-match samples. As another example, Figure 1b presents the pairs distribution over the Walmart-Amazon dataset*[2] *(10,242 pairs, 962 matches, with 60% of the entire dataset used as a train set). Again, we see a concentration of match pairs, this time mainly in a single area of the reduced latent space.*

This example suggests that a latent space can identify regions where mostly match or non-match samples reside. We use this observation to tackle a major problem in active learning, namely modeling prediction uncertainty. A standard approach is to use the model prediction confidence score as a proxy to its underlying uncertainty. However, when using transformer-based models this method is likely to fail, as they tend to produce extreme confidence values (close to 0 or 1) which barely reflect the real confidence [17, 22]. We propose a new approach for modeling the uncertainty of a pair, based on its agreement with other pairs in its vicinity, such that when the correspondence between pair's prediction and its surroundings is higher, its uncertainty becomes lower.

1.1 Contributions

In this work we offer a novel active learning approach to entity matching, utilizing an effective space partitioning for diversity, and locality-based measurements for targeted sample selection. Specifically, equipped with the observed phenomenon of concentration of match pairs (Figure 1), we propose a sampling strategy that can be intuitively demonstrated using the popular Battleship game[3] (hence the battleship approach in the manuscript title). Our strategy uses a locality principle, searching in a vicinity of a match (or non-match) pair to create a balanced training set for human labeling. To balance locality with diversity and supporting model generalization we use a constrained version of the well-known k-means clustering algorithm [6], combined with centrality and uncertainty measures that offers an efficient selection mechanism of samples.

Our main contribution can be summarized as follows:

(1) A novel active learning sample selection method, based on match and non-match pair locality to ensure a balanced training set. The proposed solution uses tuple pair representation as a tool for diverse sampling.
(2) A novel uncertainty measure, overcoming the barrier of dichotomous confidence values assigned by transformer pre-trained language model.
(3) A large-scale empirical evaluation showing the effectiveness of our approach.
(4) An open source access to our implementation.[4]

The rest of the paper is organized as follows. We survey the necessary background for our task in Section 2, positioning entity matching in the context of active learning. In Section 3 we propose a new active learning algorithm for entity matching, inspired by the battleship game. We describe the experimental setup in Section 4 and present the empirical evaluation in Section 5. Then, we

[1] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Amazon-Google/exp_data/
[2] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Walmart-Amazon/exp_data/
[3] https://en.wikipedia.org/wiki/Battleship_(game)
[4] https://github.com/BarGenossar/The-Battleship-Approach-to-AL-of-EM-Problem

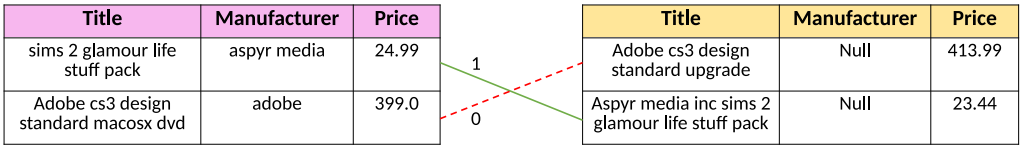


Fig. 2. An example of entity matching between two tables. Candidate pairs are connected via a line where solid green (1) is a match and dotted red (0) a non-match.

expand the discussion of the algorithm's components in Section 6. Related work is presented in Section 7 and we conclude in Section 8.

2 PRELIMINARIES

In this section we present the task of entity matching (Section 2.1) and introduce the general framework of active learning (Section 2.2).

2.1 Entity Matching

For convenience of presentation, and following [29], we introduce entity matching as the task of matching entities between two clean datasets, such that in each dataset there are no entity duplicates (clean-clean matching). This definition can be easily extended to a single dirty dataset or multiple (clean or dirty) datasets. Consider two datasets D_1 and D_2 of entities (*e.g.,* products, locations, academic papers, *etc.*). Given a pair of tuples $(r_1, r_2) \in D_1 \times D_2$, the objective of entity matching is to determine whether $r_1 \in D_1$ and $r_2 \in D_2$ represent the same real-world entity. The essence of entity matching is to generate (train) a model that can accurately provide match/non-match predictions regarding tuple pairs.

Example 2. *To illustrate, Figure 2 shows an instance taken from the Amazon-Google dataset. An example of a matching tuple pair (labeled with 1) involves the first tuple from the Amazon table (left) and the second tuple from the Google table (right). All other pairs, including the second tuple of the Amazon table and the first tuple of the Google table, are non-match pairs (labeled with 0).*

Traditionally, the matching phase is preceded by a blocking phase [43], which aims at reducing the computational effort of classifying the full set of possible pairs ($|D_1| \times |D_2|$). This is usually done by eliminating unlikely matches, keeping a relatively small set of candidate pairs. In this work we focus on the matching itself, assuming that the candidate pair set was already extracted using existing methods (see empirical comparison [44]). Each tuple is structured using a set of attributes $\{A_i\}$ and a tuple can be represented as a set of pairs $\{(Att_i, Val_i)\}$ for $1 \leq i \leq m$.

In this work, we use transformer-based pre-trained language models, such as *BERT* [9] and its descendants (*e.g.,* Roberta [32] and DistilBERT [50]), which have shown state-of-the-art results in traditional natural language processing classification tasks and have been utilized for entity matching as well [7, 29, 39]. In such language models, a special $[CLS]$ token is added at the beginning of the text, allowing the model to encode the input representation into a vector with dimensionality of 768. To yield a prediction, the embedding of the $[CLS]$ token is pooled from the last transformer layer and injected as input into a fully connected network. In the context of entity matching, we follow [29] and serialize tuple pairs with a syntactic separation between them, where each tuple by itself is a serialization of its attribute-value pairs.

Example 3. *For example, the match candidate pair in Figure 2 is serialize as follows:* "[CLS] [COL] title [VAL] sims 2 glamour life stuff pack [COL] manufacturer [VAL] aspyr media [COL] price [VAL] 24.99 [SEP] [COL] title [VAL] aspyr media inc sims 2 glamour life stuff pack [COL] manufacturer [VAL] [COL] price [VAL] 23.44".

The model uses the extracted embeddings of the [*CLS*] token to make a final prediction regarding the pair. Since it serves as the connecting thread between the entire token sequence and its final prediction, the [*CLS*] token is widely treated as the *representative vector of the input.*

2.2 Active Learning

Supervised models, particularly those based on neural networks, require massive amounts of labeled data. Active learning is widely employed in low resource settings to avoid the costly use of human annotators [38, 53]. Essentially, active learning focuses on iterative selection of samples to be labeled in a way that the selected ones are projected to provide significant informative power to the learning model. The selected samples are labeled by an oracle (*e.g., a human annotator*) and then added to the training set.

A common selection criterion in active learning is the *certainty* a model assigns with samples [25, 38, 48]. Alongside the prediction itself, most classifiers also produce confidence values, where low confidence samples are believed to be more conductive to the model understanding. This principle was also employed in entity matching and entity resolution [5, 19, 25, 48]. For example, Kasai *et al.* [25] measure pair uncertainty using conditional entropy:

$$H(p) = -plog(p) - (1-p)\,log(1-p) \tag{1}$$

where p is the confidence the model assigns a given pair being a match. Another selection criterion in active learning is the *centrality* of samples, assuming more representative data samples are more informative to the model as well [67, 69]. Centrality can computed in multiple ways (*e.g.,* betweenness centrality [11]).

In this work, we use a graph structure to model a pair set, and apply both measures, accounting for vector space spatial considerations. For uncertainty, we extend Eq. 1 to neighborhood agreement computation. As for centrality, we use *PageRank* [42], a well known measure that captures relative importance of a node in a graph by computing a probability distribution using graph topology.

3 ACTIVE LEARNING USING THE BATTLESHIP APPROACH

This section introduces an active learning algorithm to select, in an iterative manner, beneficial samples to label for solving entity matching problems. The proposed algorithm uses three selection criteria, namely *certainty*, *centrality*, and *correspondence*. The first two were introduced as general selection criteria for active learning in Section 2.2. *Correspondence* handles a unique setting of data imbalance in entity matching, with a high percentage of no match pairs. Label imbalance interferes with the production of a quality model that generalizes well to new unseen data, especially for low resources. Our preliminary experiments show that if the quantity of matching pairs is insufficiently large the model often gets stuck in a local minimum where it only predicts no match labels. A crucial challenge of active learning in this domain is therefore to balance match and no match samples, so that the model can generalize well for both classes, overcoming this cold start challenge.

The proposed sampling strategy uses a modified version of conditional entropy, expressing the induced uncertainty of a tuple pair with respect to spatial considerations, and PageRank centrality. To cope with the label imbalance we aim at pinpointing regions of the sampling space that are more likely to include match samples, in support of correspondence. Then, we use the pooled sample representations to carefully select samples. From that point on, active learning plays a role in tuning the already existing model. Table 1 summarizes the notations used throughout this work.

Table 1. Notation Table

Notation	Meaning
B	Labeling budget per active learning iteration
I	Number of active learning iterations
i	Active learning iteration index
D	dataset
D_i^{train}	Labeled subset of D in iteration i
D_i^{pool}	Unlabeled subset of D in iteration i
M_i	Trained model in iteration i
$\mathbb{E}_i\,(j,p)$	Pair representation of (r_j, r_p) in iteration i
$\phi\,(v)$	Model confidence in the pair match
$\pi\,(e)$	The weight of edge e
$G_i^+ = (V_i^+, E_i^+)$	Pair graph over match predicted samples from D_i^{pool}
$G_i^- = (V_i^-, E_i^-)$	Pair graph over non-match predicted samples from D_i^{pool}
$G_i = (V, E_i)$	Pair graph built upon D
CC_i^*	Connected components of G_i^* $(* \in \{+,-\})$
CC_i	Connected components of G_i

3.1 Overview

A complete illustration of the proposed active learning algorithm is given in Figure 3. We briefly outline the process first, and dive into details of the various components in the remainder of the section (see subsection numbering in Figure 3). Active learning algorithms are iterative, and we use $i \in \{0, 1, \cdots, I\}$ to denote the iteration number. D, the initially unlabeled dataset of candidate pairs, is partitioned in each iteration i into two disjoint pair sets, D_i^{train} and D_i^{pool}, such that $D = D_i^{train} \cup D_i^{pool}$ and $D_i^{train} \cap D_i^{pool} = \emptyset$ (bottom left of Figure 3). D_i^{train} is an already labeled subset of D while D_i^{pool} represents the remaining unlabeled dataset of D.

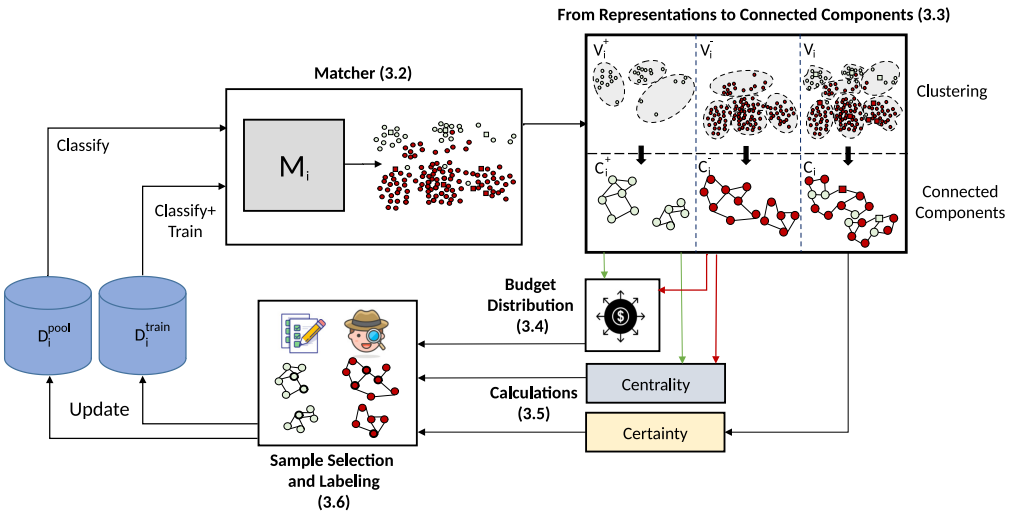


Fig. 3. An Illustration of the battleship approach framework.

In a single active learning iteration B (denotes the labeling budget) new pairs are sent for labeling. Following previous works [19, 25], we assume the existence of labeled initialization seed D_0^{train} (such that $D_0^{pool} = D \setminus D_0^{train}$), which is used for training the initial model M_0. Samples from the target domain D are first labeled with M_0 and then, through an iterative process, the labeled data in iteration i (D_i^{train}) is used to train a model M_i (mid top part of Figure 3). The entire set, D, is then inserted into the new model, yielding a representative vector for each pair. Pairs are separated into connected components, built upon constrained K-Means clustering (right top part of Figure 3). Each connected component receives a relative budget to its size and used for calculating pair certainty and centrality scores. The samples are carefully selected under budget limitations (mid bottom part of Figure 3), considering the three selection criteria (certainty, centrality, and correspondence). New labeled data are then moved from D_i^{pool} to D_{i+1}^{train} and the iterative process repeats until the halting condition is met.

3.2 Training a Matcher with Labeled Samples

At each iteration i (including the initialization phase) the model that was trained with D_{i-1}^{train} (top left part of Figure 3) is now targeted to classify pairs from the entire set D. We inject all pairs $(r_i, r_j) \in D$ through the model to extract $\mathbb{E}_i\,(j, p)$ and $\hat{y}_{j,p}$, the pair representation (the embeddings of the $[CLS]$ token, taken from the last transformer layer) and the model's prediction, respectively. With each new active learning iteration, the model is trained with enriched training set, such that new labeled samples are added to previous ones. For example, after the first active learning iteration the model is trained with D_1^{train}, which consists of D_0^{train} and B new labeled samples, selected from D_0^{pool}. As the iterative process progresses, we expect $\mathbb{E}_i$ to contain a better pair representation, expressed in a clearer separation of match and no match samples.

3.3 Pair graphs: From Spatial Representations to Connected Components

The generation of pair representations is a cornerstone in selecting the most informative samples to be labeled. We hypothesize that a major share of informativeness for the entity matching task is a derivative of regional properties. Hence, we exploit the extracted representations in a way that allows us to pinpoint regional representatives. To do so, we build a graph derived from tuple pair vector-space representations and their similarities, and partition it into connected components as a tool of excavating spatial properties. We also make use of the graph to compute certainty and centrality scores (Sections 3.5.1 and 3.5.2).

Let $G = (V, E)$ be a *pair graph* such that V is a set of nodes representing tuple pairs. Each $v \in V$ is associated with its corresponding representative vector and a value $\phi(v)$ that denotes the model confidence in the pair match. E is a set of weighted edges, where existence of an edge $e = (u, v) \in E$ reflects spatial proximity of the pairs u and v. The edge weight $\pi(e)$ is the similarity score between u and v, calculated using cosine similarity function.

3.3.1 ***Clustering Using Constrained K-Means.*** As a preparatory phase to the graph creation, we cluster the samples using K-Means over their representations. The motivation for this step is twofold. First, it partitions the vector-space into separate regions, which guarantees diverse sampling. Second, the edge creation process requires similarity comparisons between samples. Our algorithm allows comparisons only for samples that reside in the same cluster, hence significantly reducing the computational effort. While the second motivation also serves to motivate blocking in entity matching, it is the diversity that mainly drives the algorithm decision making.

We apply a constrained version of K-Means [6] to avoid small clusters that cannot be represented under budget limitations, or alternatively, large clusters that demand multiple similarity

comparisons. We set a minimal and maximal size for a cluster and select the k value according to the Kneedle algorithm [52] over the average sum of squared distance between the centroid of each cluster to its members. If the Kneedle algorithm fails to find a target value we select k as the one that maximizes the silhouette score [62], a common clustering evaluation metric measuring intra-cluster cohesiveness comparing to inter-cluster separation.

3.3.2 ***Edge Creation.*** The clustering of pair representations is employed for generating a representation graph. The structure of the graph is determined by the selection of hyperparameters defining node connectivity. The graph is utilized to capture the notions of certainty and centrality. Therefore, each node shall be connected to a minimal number of neighbors (for spatial-aware certainty calculation, as detailed in Section 3.5.1) in a way that central nodes (with large number of adjacent representations in $\mathbb{E}_i$) are rewarded by being directly connected to additional nodes.

We connect each node (pair representation) to its q (a hyperparameter of the model) nearest neighbors (in terms of cosine similarity score) among its cluster members. A large value of q leads to a more robust certainty calculation and to a better graph connectivity, reducing the possibility of obtaining multiple small connected components that are under-represented under the budget distribution policy (Section 3.4). However, it also increases the computational effort and might lead to a misrepresentation of cluster margins, which can harm the diversity. In addition, and for the sake of a better centrality estimation, we sort the remaining pairs within each cluster in descending order based on their similarity scores. We then generate edges for the closest node pairs in this sorted list. The ratio of selected node pairs is defined by a hyperparameter such that the total number of additional edges is proportional to the cluster size. Intuitively, a more central node is more likely to be connected to a larger number of nodes. We assign an edge e with its corresponding similarity score, denoted $\pi(e)$, as its weight. The resulting graph consists of multiple connected components, where each cluster yields one (or more) connected components.

Example 4. *Figure 4 and Table 2 jointly offer an illustrative example of the edge creation process. Assume that* $s1$-$s8$ *are pair representations forming a single cluster, created according to the principles outlined in Section 3.3.1. The samples* $s1, s2, s3$ *and* $s4$ *(light green circles) were predicted as match by the matcher (see Section 3.2),* $s5$ *and* $s6$ *(red circles) were predicted as no match and* $s7$ *and* $s8$ *were already labeled as match and no match, respectively, in a previous iteration. The values shown in Table 2 reflect the similarity scores between pair representations, while the values in the diagonal (blue cells) are the confidence scores assigned by the model (already labeled samples receive a confidence score of* 1*), e.g., the similarity score between* $s1$ *and* $s2$ *is* 0.9 *(relatively high as reflected by their closeness in Figure 4) and the matcher confidence of* $s1$ *being a match is* 0.95*. Similarity score is symmetric, leading to a symmetric matrix. Assume that in this example* $q = 2$*, namely each pair is automatically connected to its two nearest neighbors. In addition, assume that* 0.15 *of the remaining potential edges are created (the closest ones among them).*

Each pair is connected to at least two other pairs (its two nearest neighbors), denoted by solid lines. A pair can be connected to other pairs by more than two solid lines. For example, $s1$ *is automatically connected to* $s2$ *and* $s7$*, since they have the highest scores in its similarity list (with* 0.9*).* $s1$ *is also connected by a solid line to* $s8$ *since the former is among the two nearest neighbors of the latter (although* $s8$ *is only the fifth closest pair to* $s1$*). The total possible number of edges in this example is* $\binom{8}{2} = 28$*, and* 12 *edges are created in the first stage of connecting each pair to its two nearest neighbors. Therefore,* 16 *possible edges are remaining, such that two (*$\lfloor 0.15 \cdot 16 \rfloor = 2$*) additional edges shall be created. We rank those edges in a descending order, resulting in linking* $s1$ *to* $s5$ *(*0.85*) and* $s5$ *to* $s7$ *(*0.63*), denoted in the figure by dashed lines. It is noteworthy that we do not directly connect two labeled pairs, as they are not a target for the certainty calculations (Section 3.5.1). Thus,* $s7$ *is not connected to* $s8$ *even though their similarity is higher than the similarity between* $s5$ *and* $s7$*.*

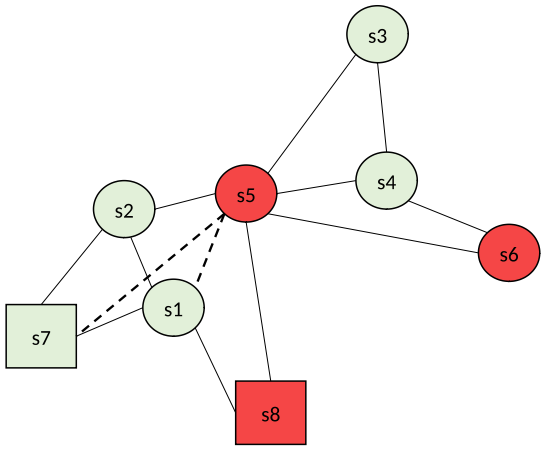


Fig. 4. Edge creation illustration: Light green circles and squares represent match predicted and labeled samples, respectively. Red circles and squares represent no match predicted and labeled samples, respectively.

3.3.3 ***Prediction and Heterogeneous Graphs.*** To support effective sample selection that caters to the class imbalance problem, we suggest to separate match from no match samples. To do so, we create three graphs using the clustering mechanism described in Section 3.3.1, each with a different set of nodes. In each iteration i, we partition D_i^{train} into match labeled samples (D_i^+) and no match labeled samples (D_i^-). In a similar fashion, we also partition D_i^{pool} into $D_i^{pool+} = \{r, r' \in D_i^{pool} \mid M_{i-1}(r, r') = 1\}$ and $D_i^{pool-} = \{r, r' \in D_i^{pool} \mid M_{i-1}(r, r') = 0\}$, using the latest model to predict the labels of the unlabeled samples. Next, we create the three pair representation graphs $G_i^+ = (V_i^+, E_i^+)$, where $V_i^+ = D_i^{pool+}$ includes only the samples predicted to match, $G_i^- = (V_i^-, E_i^-)$, where $V_i^{pool-} = D_i^-$ contains samples predicted to be no match, and $G_i = (V, E_i)$, where $V = D$ contains all samples. Edges are generated, as detailed in Section 3.3.2.

The first two graphs are prediction-based graphs, used for spatial sampling (Section 3.4) and centrality calculations (Section 3.5.2). The third is a heterogeneous graph, consisting of both labeled and unlabeled samples, disregarding their labels and predictions. It is used for certainty calculations (Section 3.5.1). We denote by CC_i^+, CC_i^-, and CC_i the connected components sets that are generated from the three graphs and show how such a separation enhances the chances of successful sample selection.

Table 2. Similarity scores for the samples presented in Figure 4. The values (blue cells) are the confidence scores assigned by the matcher for each sample.

	s1	***s2***	***s3***	***s4***	***s5***	***s6***	***s7***	***s8***
s1	*.95*	*.9*	*.5*	*.6*	*.85*	*.5*	*.9*	*.82*
s2		*.92*	*.55*	*.58*	*.92*	*.45*	*.83*	*.6*
s3			*.96*	*.75*	*.67*	*.56*	*.4*	*.38*
s4				*.94*	*.88*	*.84*	*.5*	*.55*
s5					*.98*	*.57*	*.63*	*.65*
s6						*.88*	*.41*	*.54*
s7							*1*	*.64*
s8								*1*

Example 5. *In the illustration shown in Figure 3 (mid top part of the figure) we use light green and red circles to denote match and no match predicted samples, respectively, and squares for already labeled samples (with the same color scheme). The squares play a role only in the heterogeneous graph (right top part of the figure), as only match and no match predicted samples appear in the graphs derived from V_i^+ and V_i^-, respectively.*

3.4 Budget Distribution

In a single active learning iteration, B unlabeled pairs are selected to be labeled. We aim at a balanced pair selection, selecting both match and no match samples and sampling proportionally from dense and sparse spatial regions. Towards this end, we make use of the connected components that are created according to the description in Section 3.3 (see right bottom part of Figure 3).

The match and no match predicted samples form separate connected component sets, CC_i^+ and CC_i^-. Let B be the *labeling budget*, split into two distinct budgets, B^+ and B^- ($B = B^+ + B^-$), for expected match and expected no match pairs, respectively. Intuitively, we would like to assign larger share of the budget to larger connected components. We define the budget share of a connected component cc as follows:

$$cc_{i,budget} = \left\lfloor \frac{B^* \cdot |cc_i|}{\sum_{cc_i' \in CC_i^*} |cc_i'|} \right\rfloor \tag{2}$$

where the asterisk in B^* and CC_i^* stands for either + or −, and $|cc_i|$ is the size of the connected component. Since we round down the budgets (being a positive integer), any budget residue is randomly distributed among connected components.

Example 6. *Assume that 3,000 samples were predicted as matching, split into 10 connected components, such that two of them consist of 500 samples each, four of 300 samples each and another four of 200 samples each. In addition, assume that $B^+ = 50$. According to the budget distribution policy of Eq. 2 the first two (500 samples each) will be assigned with a budget of 8 ($\lfloor \frac{50 \cdot 500}{3000} \rfloor$) each, the next four with a budget of 5 each ($\lfloor \frac{50 \cdot 300}{3000} \rfloor$) and the last four with a budget of 3 each ($\lfloor \frac{50 \cdot 200}{3000} \rfloor$). The residue (2) is randomly allocated.*

3.5 Selection Criteria Calculations

Recall the three criteria for sample selection, namely certainty, centrality, and correspondence. Correspondence is supported by the generation of the graphs G_i^+ and G_i^- (Section 3.3.3) and the balanced budget distribution between match and no match connected components (Section 3.4). We focus next on certainty and centrality when selecting samples to be labeled (see bottom part of Figure 3), adapting them to the unique needs of entity matching.

3.5.1 ***Certainty.*** Conditional Entropy (Eq. 1) is one of the common methods for measuring certainty in active learning [19, 25]. However, highly parameterized models such as transformer-based pre-trained language models tend to produce an uncalibrated confidence value [17, 22], assigning mostly dichotomous values close to either 0 or 1. Such values provide little entropy differentiation, rendering pure measures such as conditional entropy unreliable for the selection mechanism. To overcome the dichotomous barrier, we propose to add a spatial interpretation to certainty by computing the disagreement of a pair with other pairs in its vicinity, using $G_i = (V, E_i)$ (Section 3.3.3) and its induced connected components CC_i for an effective certainty computation.

Let $G_{cc} = (V_{cc}, E_{cc})$ be a subgraph of G_i where V_{cc} is the set of nodes in $cc \in CC_i$, each representing a candidate pair, and E_{cc} is a set of weighted edges labeled with the cosine similarity of the corresponding pair representations. V_{cc}^+ and V_{cc}^- represent the pair set predicted to be, or already

labeled as match and no match, respectively. Given a node $v \in cc \cap D_i^{pool}$, $N^+(v)$ denotes the set of nodes in V_{cc}^+ that are connected to v and $N(v)$ denotes the entire set of nodes connected to v. The spatial confidence of a model M_{i-1} regarding $v \in cc$ is the weighted average confidence of v's neighbors with respect to its assigned prediction, defined as follows:

$$\tilde{\phi}(v) = \frac{\sum_{v' \in N^*(v)} \pi(v, v') \cdot \phi(v')}{\sum_{v' \in N(v)} \pi(v, v') \cdot \phi(v')} \tag{3}$$

where the asterisk is + if the prediction is 1, otherwise it is −, $\pi(v, v')$ is the cosine similarity between v and v' representations, and $\phi(v')$ represents the confidence in a label and is set to 1 for all $v' \in D_i^{train}$. For all other nodes, not in the training set, it is set to the confidence value assigned by M_{i-1} to v'.

Example 7. *Consider again Figure 4 together with Table 2 and assume that we are interested in calculating $\tilde{\phi}(s1)$. $s1$ is predicted a match, hence the numerator consists of its match predicted or labeled neighbors ($s2$ and $s7$), while the denominator considers also the no match nodes ($s5$ and $s8$). By using the similarity and confidence scores of Table 2, the desired score is:*

$$\tilde{\phi}(s1) = \frac{0.9 \cdot 0.92 + 0.9 \cdot 1}{0.9 \cdot 0.92 + 0.9 \cdot 1 + 0.85 \cdot 0.98 + 0.82 \cdot 1} = 0.51$$

Relying on our observation that regions in the latent space tend to be homogeneous, we employ Eq. 1 and define the *spatial entropy* of a node $v \in cc$ as $H\left(\tilde{\phi}\right)$. We combine both node's model-based prediction confidence and its spatial confidence, computing the final certainty score as a linear combination of the standard conditional entropy and the spatial entropy (both using Eq. 1), as follows:

$$\mathbb{S}_{unc}(v) = \beta \cdot H(\phi(v)) + (1 - \beta) \cdot H\left(\tilde{\phi}(v)\right) \tag{4}$$

where β is a weighting parameter ($0 \leq \beta \leq 1$).

3.5.2 ***Centrality.*** We use PageRank [42], a well-known centrality measure for node's importance in a graph, originally used for Web retrieval. PageRank outputs a probability vector over the entire set of nodes, such that the higher the probability is, the more central is a node. Since edge directionality is important for PageRank, we produce two inversely directed edges for each edge in a connected components with the same edge weight (similarity score, see Section 3.3.2).

Centrality is computed only over the available pool elements, CC_i^+ and CC_i^-, disregarding the labeled samples. PageRank centrality of a node $v \in V_{cc}$ is calculated as follows:

$$\mathbb{S}_{cen}(v) = \rho \sum_{v' \in N(v)} A_{v,v'} \frac{\mathbb{S}_{cen}(v')}{\sum_{v'' \in V_{cc}} A_{v',v''}} + \frac{1 - \rho}{|V_{cc}|} \tag{5}$$

where A is a weighted adjacency matrix and ρ is a sampling parameter, traditionally used in PageRank to avoid dead-end situations [42].

3.6 Sample Selection and Labeling

To overcome possible scaling issues, we propose to rank pairs (nodes) in a descending scores order, according to both certainty and centrality. We define $\mathfrak{R}_{cer}(v)$ and $\mathfrak{R}_{cen}(v)$ as the ranking of node v according to its certainty and centrality score, respectively. Then, we weigh the overall ranking, as follows:

$$\alpha \cdot \mathfrak{R}_{unc}(v) + (1 - \alpha) \cdot \mathfrak{R}_{cen}(v) \tag{6}$$

where α is a weighting parameter ($0 \leq \alpha \leq 1$), of which we expand the discussion in Section 6.2. For a connected component cc, the top $cc_{i,budget}$ pairs according to the weighted rank are selected to be labeled, removed from the D_{i+1}^{pool}, and inserted into D_{i+1}^{train}, ready to train the model for the next iteration. Similar to previous works [19, 25, 49, 53], we assume the existence of a perfect labeling oracle, recognizing that in real-world settings a labeler might be exposed to biases that affect labeling accuracy [56].

3.7 Optimization with Weak Supervision

In addition to the new labeled samples obtained in each iteration, we enrich the training set without exceeding the labeling budget B. To do so, we use a weak supervision approach, where unlabeled samples are augmented into the training set with their corresponding model-based prediction, treated as a label. By that, we allow the model to learn from a larger training set without using human labor for annotation. *Kasai et al.* [25] implemented this approach by selecting the most confident pairs, namely those with the lowest conditional entropy values (Eq. 1). Following the principles presented in Section 3.5.1, we define the most confident pairs in a spatial-aware fashion, namely the selected samples are those that minimize the value of Eq. 4. To enhance diversity of sampling, the label-wise budget ($\frac{B}{2}$ for each of match and no match predicted sample sets) is distributed over the connected component (Section 3.3) using the same procedure in Section 3.4. As this addition was proven to be effective in our preliminary experiments, we have incorporated it into the entire set of experiments.

4 EXPERIMENTAL SETUP

In this section we detail the benchmarks and evaluation methodology used to asses the performance of the battleship approach.

4.1 Datasets

We use six publicly available datasets, from different domains. To be consistent with recent works [26, 29], we assume that a set of candidate tuple pairs is given, possibly a result of a blocking phase. A summary of the datasets (referring to the complete training set sizes) is given in Table 3 and detailed next.

Walmart-Amazon and Amazon-Google: The datasets of Walmart-Amazon and Amazon-Google, taken from the Magellan data repository [26],[5] are well-known for evaluation of entity matching solutions. We use the training, validation and test sets, with the ratio of 3:1:1 provided by the benchmark used in previous works (*e.g.,* [29, 41]). The relative part of matching pairs (see Table 3) is fairly small, turning the initialization of the model into a challenging task.

WDC Cameras and WDC Shoes: The web data commons (WDC) dataset [46] contains product data extracted from multiple e-shops[6] and split into four categories. Following [29], we use only the product title, ignoring other attributes. We focus on the medium size datasets and the categories of Shoes and Cameras. We keep the same partition as [29], with a test set of $\sim 1,100$ pairs per category dataset, while the remaining pairs are split into training and validation with the ratio of 4:1.

ABT-Buy: ABT-Buy is also a product dataset. Unlike the rest of the datasets, which are all structured, ABT-Buy contains long textual inputs. The task is to match company homepages to Wikipedia pages describing companies. We use the same 3:1:1 partiotion of the data, as used by [41].

DBLP-scholar: DBLP-scholar is also a widely used benchmark for entity matching, containing bibliographic data from multiple sources. Whereas DBLP and is a high quality dataset, Google

[5]https://github.com/anhaidgroup/deepmatcher/blob/master/Datasets.md
[6]http://webdatacommons.org/largescaleproductcorpus/v2/index.html

Table 3. Statistics of the datasets used in our experiments. The size values refer to the training sets.

Dataset	Size	%Pos	#Atts
Walmart-Amazon	6,144	9.4%	5
Amazon-Google	6,874	10.2%	3
Cameras	4,081	21.0%	1
Shoes	4,505	20.9%	1
ABT-Buy	5,743	10.7%	3
DBLP-Scholar	17,223	18.6%	4

scholar is constructed using automatic Web crawling, hence lacking a rigorous data cleaning process. this benchmark also comes with a predefined training, validation and test sets, with the ratio of 3:1:1 [26].

4.2 Implementation Details

Experiments were performed on a server with 2 Nvidia Quadro RTX 6000 and a CentOS 7 operating system. Our implementation is available in a git repository.[7]

Model and Optimization: We adopted the publicly available[8] implementation of DITTO [29], and modified it to yield, alongside predictions, also pair representations and confidence values. In each active learning iteration $i \in \{0, 1, \cdots, I\}$ we train a model with the updated training set D_i^{train} (see Section 3.2). For the first five datasets (Table 3) we set the number of epochs to 12, while for the sixth (DBLP-Scholar) we run only over 8 epochs. These number were selected after preliminary experiments showing no significant improvement (if any) is achieved by using a larger number of epochs. The parameters of DITTO in an active learning iteration are initialized without using the values of previous iterations, and set according to the best F1 score achieved on the validation set. We use a maximum input length of 512 tokens, the maximal possible length for BERT-based models [9], and a batch size of 12. The model is trained with $AdamW$ optimizer [35] with a learning rate of $3e-5$.

The focus of this work is on the active learning's selection mechanism rather than the neural network architecture. Therefore, we use the basic form of DITTO, without optimizations [29], fine-tuning a pre-trained language model (RoBERTa [32]) on the specific task. For each configuration we report the average F1 values, calculated over 3 different seeds.

Active Learning and Pair Selection: We run 8 active learning iterations for all datasets, where B, the pair labeling budget per iteration, is fixed at 100, as well as the weak labels budget (see Section 3.7). Similar to previous works [19, 25], we start with labeled initialization seed D_0^{train}, consists of $\frac{|B|}{2}$ (in our case 50) samples for both matching pairs and non-matching pairs. Following the principles presented in Section 3, we aimed at equipping the model with a balanced set of pairs. Since match labels are harder to discover, especially in the initial active learning iterations (Section 1), we set the positive budget B^+ as $B \cdot max\left\{0.8 - \frac{1}{20} \cdot i, 0.5\right\}$, where i is the active learning iteration number. By that, we increase the chances of feeding the model with relatively large share of positive samples, contributing to its generalizability, especially in the early iterations.

A pair is represented with a 768 dimensional vector, pooled of the last hidden layer of the model. The size of a cluster (Section 3.3.1) ranges from 0.05 to 0.15 of the number of samples against which the graph is created ($\left|V_i^+\right|$, $\left|V_i^-\right|$ or $|V|$). The nearest neighbors calculations (Section 3.3.2) are implemented with FAISS library [23]. The graphs are created such that each node is connected to

[7]https://github.com/BarGenossar/The-Battleship-Approach-to-AL-of-EM-Problem
[8]https://github.com/megagonlabs/ditto

its 15 nearest neighbors. In addition, the top 3% of the rest of the sample pairs are also connected (Section 3.3.2).

4.3 Baselines Methods

We compare the battleship approach with several baseline methods to active learning of the entity matching problem. We report the results over the same test set for all approaches.

- **Random:** A naïve baseline where samples are randomly drawn from the available pool, considering neither the predictions of the model nor the benefits of pair representations.
- **DAL** (Deep Active Learning) [25]: In each active learning iteration, $\frac{B}{2}$ no match predictions and $\frac{B}{2}$ match predictions are labeled. Selected samples are the most uncertain (those maximizing the value of Eq. 1). In addition, DAL uses weak-supervision mechanism, augmenting the training set with $\frac{k}{2}$ match and no match high-confidence samples, with their assigned prediction (see Section 3.7). In the absence of implementation, we implemented a version of DAL according the to guidelines presented in [25], without the adversarial transfer learning component since source domain data is not available in our settings. To align DAL with our proposed approach, we train the model as described in Section 3.2, using DITTO [29] as the neural network matcher.
- **DIAL** (Deep Indexed Active Learning) [19]: DIAL offers an approach of co-learning embeddings to maximize recall (for blocking) and accuracy (for matching) using index-by-committee framework. Pre-trained transformer language models are used to create tuple representations, employed for vector-space similarity search to produce candidate pairs set. Then, pair selection is performed using uncertainty sampling. This characteristic differentiate DIAL from other approaches, as they are given a fixed set of candidate pairs. We test DIAL with the published implementation,[9] starting with 128 labeled samples, equally divided into matches and no matches. In our experiments, we do not evaluate DIAL on the WDC product datasets, which do not use a structure of two tables, a required input for the blocker.

In addition, we compare our approach to two methods that represent the extremes in terms of labeling resources, with the purpose of showing our approach to be competitive over the whole spectrum of resource possibilities.

- **ZeroER** (Entity Resolution using Zero Labeled Examples) [65]: ZeroER is an unsupervised approach that relies on the assumption that similarity vector for match pairs should differ from that of no match pairs. A core difference between their work and ours is that they build model-agnostic feature vector to capture tuple pairs similarity, while we extract the representation yielded by a limited resource trained model. As the WDC datasets do not fit into the required input type of ZeroER, we do not evaluate this method on these datasets.
- **Full D:** We train DITTO over the complete training set (see Table 3), assuming no lack of resources.

5 EMPIRICAL EVALUATION

We provide a thorough empirical analysis of the battleship approach. We start by comparing its performance to the multiple baselines (Section 5.1) and continue with running time report (Section 5.2).

[9] https://github.com/ArjitJ/DIAL

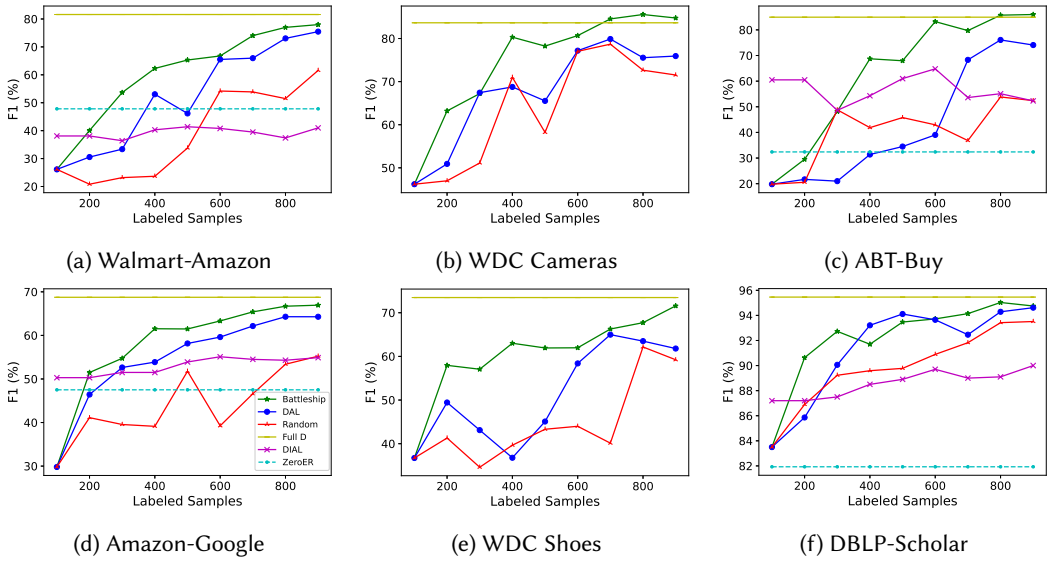


Fig. 5. Performance in terms of F1 (%) vs cumulative number of labeled samples. The Full D line represents the performance with completely available training data (the datasets sizes are displayed in Table 3.

5.1 Battleship and Baselines Performance

Figure 5 shows the F1 score with respect to the number of labeled samples for all datasets. The results reported here for the battleship approach are the average performance for the model with three values of α (0.25.0.5, 0.75), which determines the relative weight between certainty and centrality for the overall ranking (Eq. 6). For each of these α values we fixed β, which determines the relative weight between local and spatial entropy in the certainty score (of which we expand the discussion in Section 6.1), at 0.5 .

For Walmart-Amazon and Amazon-Google, the battleship approach outperforms other active learning baselines, performing almost as good as the Full D model, albeit with less labeled samples. The battleship approach beats its best baseline (DAL) by a margin of 3.3% for Walmart-Amazon and 4.1% for Amazon-Google at the end of the active learning process (900 labeled samples). For both datasets the performance of the battleship approach significantly increases during the first iterations, with a decreasing improvement rate later. The battleship approach focuses on spotting match pairs, particularly in the early stages of the iterative active learning process. Since the number of match pairs in both datasets is limited (see Table 3), the battleship approach consumes them early on, adding more no match pairs as the process progresses, which undermines the principle of a balanced pool.

The battleship approach outperforms the baselines over WDC (both cameras and shoes) as well. Furthermore, it also outperforms the Full D model (84.76 *vs.* 83.65) for the cameras dataset. A possible explanation to the success of the battleship approach over these two datasets might be their relative high ratio of matching pairs, which helps the selection mechanism to obtain balanced sampling throughout the training phase. For ABT-buy, the battleship approach also surpasses the Full D model (85.99 *vs.* 84.95), in addition to the baselines (74.08 for DAL). DBLP-Scholar is the only dataset of which the battleship approach is almost tied with the best baseline (DAL) (94.75 *vs.* 94.62), both trail by a small margin behind the Full D model (95.46).

Table 4. F1 values for varying labeled samples set size.

Type	Model	#Labels	Walmart Amazon	Amazon Google	WDC Cameras	WDC Shoes	ABT-Buy	DBLP Scholar
Unsupervised	ZeroER[65]	0	47.82	47.51	-	-	32.39	81.93
Supervised	Full D	$\|D\|$	81.60	68.75	83.65	73.48	84.95	95.46
Active Learning	Random	500	33.79	51.77	58.22	43.31	45.79	89.78
		900	61.57	55.23	71.54	59.23	52.42	93.51
	DAL[25]	500	46.17	58.15	65.53	45.08	34.49	**94.11**
		900	75.47	64.28	75.93	61.80	74.08	94.62
	DIAL[19]	500	41.40	53.90	-	-	61.30	88.90
		900	41.00	54.90	-	-	52.30	90.00
	Battleship	500	**65.30**	**61.48**	**78.24**	**61.93**	**67.95**	93.47
		900	**77.98**	**66.94**	**84.76**	**71.57**	**85.99**	**94.75**

For all datasets, the battleship approach improves upon the baseline early on, which we attribute to balance sampling that enables overcoming the cold-start problem for low-resource entity matching tasks. It can also be seen the the battleship approach requires at most two iterations to surpass the unsupervised approach of ZeroER. We note that the reported results for DITTO [29] (here as the Full D model), DIAL [19] and ZeroER [65] differ from the ones reported in the respective papers. For the former, we used the publicly available code without optimizations. For the second, we used the code provided by the authors[10] and followed the instructions in the paper to reproduce it in our setting. As for the latter, we report the results we obtained by using publicly available code[11] only over the test set, whereas the those reported in the paper [65] refer to the training, validation and test sets combined.

Table 4 emphasizes the effectiveness of the battleship approach under low resource limitation. In this table it is compared, alongside active learning baselines, with the fully trained model and with unsupervised approach, namely ZeroER. The table shows F1 performance with 500 (after four active learning iterations out of eight) and 900 (last active learning iteration) labeled samples for active learning methods. Except for DBLP-Scholar, the battleship approach beats the active learning baselines with both 500 and 900 labeled samples, while for the second it also beats the fully trained model over WDC Cameras and ABT-Buy. ZeroER, although does not rely on labeled samples at all, performs better than the baselines with 500 samples over the Walmart-Amazon dataset.

In Table 5 we estimate the performance of the battleship approach over the learning course using the Area Under Curve measurement [1, 57], calculated against the F1 plot. We notice, again, the significant gap between the battleship approach and the active learning baselines, as it is the most dominant method over all dataset. It is also noteworthy that the battleship approach beats its best competitor (DAL) over the DBLP-Scholar, despite it is the only dataset in which the second outperforms the first in terms of F1 with 500 samples. The reason for that is the preferable performance of the battleship approach against DAL during the first iterations.

5.2 Runtime Analysis

The graph in Figure 6 illustrates the average running time (seconds) for the battleship approach with a fixed β value (.5) and varying α values in the set $\{0.25, 0.5, 0.75\}$, across different datasets, as a function of the iteration number. DBLP-Scholar was excluded due to its significant variation in scale, ranging from 430 to 549 seconds per iteration, that obscures the clarity of the figure. As

[10]https://github.com/ArjitJ/DIAL
[11]https://github.com/ZJU-DAILY/ZeroMatcher

Table 5. AUC for the F1 plots.

Model	Walmart Amazon	Amazon Google	WDC Cameras	WDC Shoes	ABT-Buy	DBLP Scholar
Random	304.86	353.32	514.56	353.14	326.73	720.13
DAL	418.46	444.19	546.33	410.55	338.88	732.70
DIAL	313.45	423.70	-	-	454.30	708.50
Battleship	**491.15**	**473.03**	**605.25**	**490.06**	**515.96**	**740.54**

observed, the runtimes decrease as the active learning process progresses. This is attributed to the major impact of K-Means clustering on the overall runtime. As the active learning process continues, the pool of available data for generating the predicted match and non-match graphs (as discussed in Section 3.3.3) becomes smaller. Consequently, the K-Means computations are performed on reduced pair set, leading to improved efficiency.

In our analysis, we observed that DAL exhibits considerably faster performance, taking only a few seconds per iteration. This efficiency can be attributed to the absence of spatial considerations in this method, as the K-Means clustering step consumes the majority of the running time in the battleship approach. While there are methods such as LSH [16] and HNSW [37] that can reduce the computational effort of K-Means through approximate nearest neighbor searches, we focuse our attention on other aspects in this study and aim to explore this aspect in future work.

Additionally, we witnessed in the preliminary experiments that the number of nearest neighbors per node and the remaining pairs ratio (Section 3.3.2) also influence the running time. Reducing these parameters may lead to a tradeoff since they directly impact the centrality and certainty computations in the battleship approach. While optimizing the running time is essential in real-world applications, we need to balance it with the effectiveness of the algorithm's core components to achieve accurate results.

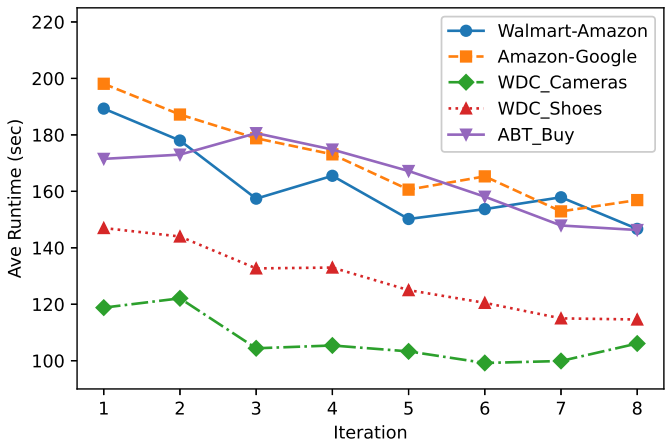


Fig. 6. Runtime of the battleship approach *vs.* iteration.

6 COMPONENTS ANALYSIS

In this section we focus on analyzing the various design choices described in Section 3, and how they impact the overall algorithm performance. Section 6.1 focuses on the local *vs.* spatial certainty tradeoff. The contribution of centrality and certainty is analyzed in Section 6.2 and the correspondence effect is elaborated upon in Section 6.3. Section 6.4 delves into the impact of the weak supervision component. Then, we conclude the discussed tradeoffs in Section 6.5.

6.1 Local *vs.* Spatial Certainty

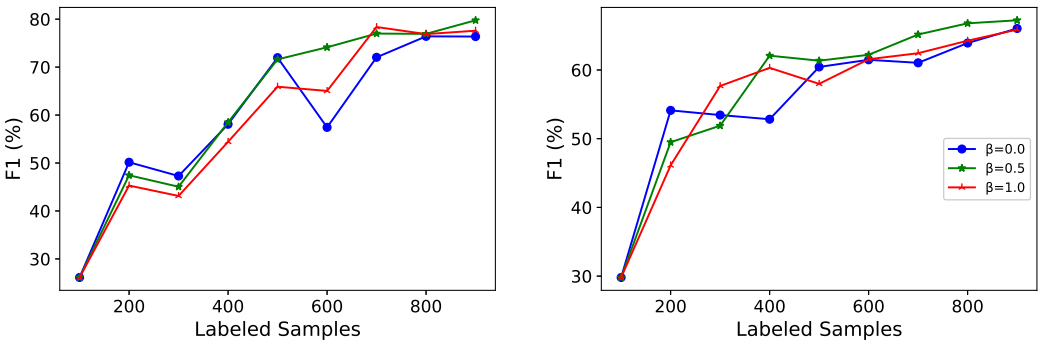
To assess the impact of local and spatial certainty, we ran experiments with $\beta \in \{0, 0.5, 1\}$ (see Eq. 4). Due to space limitations we present here only the results for Walmart-Amazon and Amazon-Google. For all three values of β we fixed α at 0.5.

Figure 7 displays the F1 performance as a function of the labeled set size. In both datasets the model obtained with $\beta = 0.5$ performs better than the other two values when the number of labeled samples surpasses 500. This implies that the fusion between local confidence score, assigned by the model, and spatial confidence, calculated with respect node's neighbors in the graph (Eq. 3) captures better model certainty. For Walmart-Amazon, the fused version ($\beta = 0.5$) reaches an F1 score of 79.76, while for $\beta = 0$ and $\beta = 1$ scores are at 76.37 and 77.59, respectively. For Amazon-Google, with $\beta = 0.5$ training ends with F1 of 67.23, outperforming the F1 scores of 66.04 (for $\beta = 0$) and 65.87 (for $\beta = 1$).

Both $\beta = 0$ and $\beta = 1$, although addressing a single aspect each, are still preferable over DAL (Figure 5), which is the best baseline. An interesting observation is that the models trained with $\beta = 0$ and $\beta = 1$ tend to fluctuate more often than the fused version, demonstrating the robustness of the fused approach.

6.2 Criteria Weighting Analysis

The sample selection mechanism relies, alongside correspondence, on a weighted ranking of certainty (Section 3.5.1) and centrality (Section 3.5.2), using the balancing factor α (see Eq. 6). In addition to three different α values (0.25, 0.5 and 0.75), which offer different combinations of both criteria, we evaluated the performance of two sub-versions of the battleship approach, namely



(a) Walmart-Amazon (b) Amazon-Google

Fig. 7. F1 performance *vs.* cumulative number of labeled samples for different β values. When $\beta = 0$ the certainty is calculated only with respect to the spatial confidence (Eq. 3), and when $\beta = 1$ only by the model confidence.

Table 6. F1 values after the last iteration with various α values ($\beta = 0.0$).

Dataset	$\alpha = 0.0$	$\alpha = 0.25$	$\alpha = 0.5$	$\alpha = 0.75$	$\alpha = 1.0$
Walmart-Amazon	77.71	78.04	**79.76**	76.14	76.13
Amazon-Google	65.1	65.38	67.23	**68.22**	66.10
WDC Cameras	83.85	**86.53**	84.97	82.79	82.22
WDC Shoes	66.08	68.48	72.98	**73.24**	71.65
ABT-Buy	83.21	86.07	84.31	**87.59**	81.52
DBLP-Scholar	93.95	94.47	**96.03**	93.75	93.81

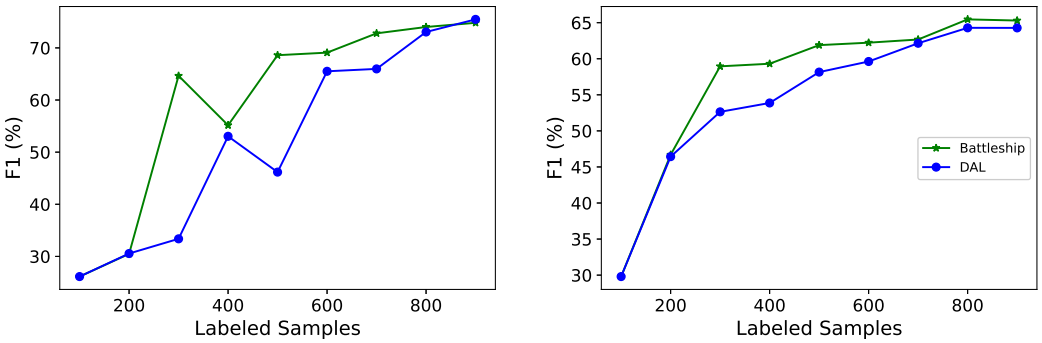
Battleship (cen) and *Battleship (unc)*, with $\alpha = 0.0$ and $\alpha = 1.0$, respectively (see Section 4.3). For all experiments, and following Section 6.1, we fixed β at 0.5.

Table 6 displays the F1 average scores (after the final iteration) for the different datasets using the various α values. For all datasets, the model has the best performance for $\alpha \in \{0.25, 0.5, 0.75\}$, *i.e.*, taking into account both criteria. This implies that both local centrality and model's confidence are important factors in identifying informative samples that can complement the model training. It is noteworthy that in most cases, the results obtained with $\alpha \in \{0, 1\}$ outperforms DAL, the most competitive active learning baseline. This suggests that the correspondence criterion, handled by the graph structure and budget distribution, also plays a key role in the sample selection mechanism, as discussed in Section 6.3.

6.3 The Correspondence Effect

To quantify the contribution of the correspondence criterion, although not directly affected by Eq. 4, we fixed the values $\alpha = 1$ and $\beta = 1$. In this case, the certainty score is determined only by the confidence assigned by the model, and the final ranking is performed only according to the certainty criterion (Eq. 6). These conditions dictate the same selection mechanism as the one proposed by DAL, with the exception that in our approach samples are confined to their connected component (Section 3.3.3). In other words, with these parameters the selection of DAL is used over connected component.

Figure 8 provides evidence of the effectiveness of vector-space partitioning and budget distribution policy. In the case of the Amazon-Google dataset, the battleship approach consistently outperforms DAL throughout the active learning process. For the Walmart-Amazon dataset, the



(a) Walmart-Amazon (b) Amazon-Google

Fig. 8. F1 performance *vs.* cumulative number of labeled samples for $\alpha = 1, \beta = 1$.

battleship approach achieves an F1 value of 74.81 after 8 active learning iterations, slightly lower than DAL's (75.47). However, the battleship approach demonstrates notably superior performance in terms of AUC scores (485.20 vs 418.46). This is attributed to the fact that the battleship approach consistently leads over DAL in previous iterations, with DAL only surpassing in the final iteration.

6.4 Weak Supervision Impact

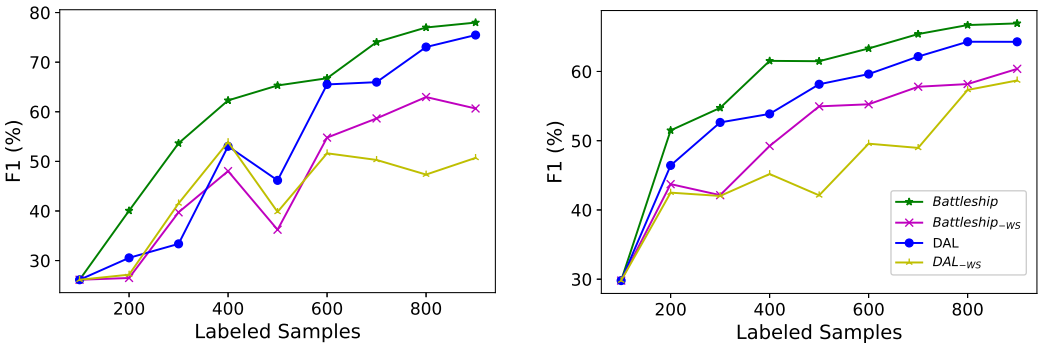
We conducted a comprehensive examination to investigate the contribution of the weak supervision component (Section 3.7) on our proposed battleship approach. Specifically, we assess the effectiveness of incorporating weak supervision within both the battleship approach and DAL. Furthermore, we compare our weak supervision approach, which identifies the most confident pairs based on the principles outlined in Section 3.5.1, with the weak supervision approach employed in the baseline method DAL.

6.4.1 Weak Supervision Impact. Figure 9 shows the performance of the battleship approach (average scores computed over $\{0.25, 0.5, 0.75\}$, as reported in Section 5.1) and DAL with and without weak supervision ($Battleship/DAL$ and $Battleship_{-WS}/DAL_{-WS}$, respectively).

For the Walmart-Amazon dataset, incorporating weak supervision into the battleship approach ($Battleship_{ws}$) yields progressively higher scores as the number of labeled samples increases, peaking at 77.98. Conversely, the battleship approach without weak supervision ($Battleship_{-WS}$) demonstrates variable performance, with a maximum score of 60.66. Likewise, in the case of DAL, removing weak supervision (DAL_{-WS}) results in decreased scores, reaching a maximum of 50.7. A similar pattern is observed in the Amazon-Google dataset, where the inclusion of weak supervision leads to consistent improvement within the battleship approach, achieving a maximum score of 66.94. In contrast, $Battleship_{-WS}$ exhibits more fluctuation, with a maximum score of 60.37. Similarly, DAL demonstrates a comparable trend, with weak supervision contributing to steady performance enhancement (64.28), while its absence leads to more variable results (58.7).

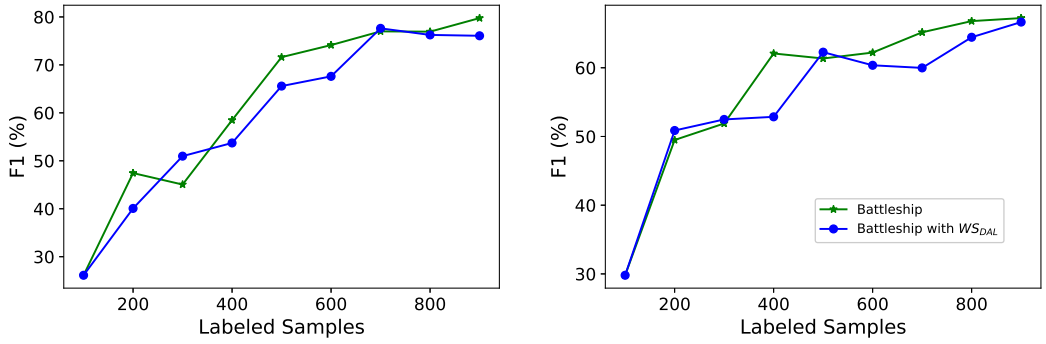
These results imply that weak supervision is effective in enhancing the performance of both the battleship approach and DAL, leading to more stable results over learning course.

6.4.2 Weak Supervision Method Comparison. As highlighted in Section 3.7, DAL utilizes a weak supervision method that focuses on identifying pairs where the model exhibits high confidence, as indicated by minimizing Eq. 1. In our battleship approach we integrate between the model's standard conditional entropy and spatial entropy using Eq. 4. Figure 10 presents the performance



(a) Walmart-Amazon (b) Amazon-Google

Fig. 9. F1 scores with and without weak supervision.



(a) Walmart-Amazon (b) Amazon-Google

Fig. 10. F1 performance of the battleship approach and the battleship approach with DAL's weak supervision.

of the battleship approach (with $\alpha = 0.5, \beta = 0.5$) over two cases, differing only in their weak supervision method, such that *battleship* uses Eq. 4, while *battleship* with WS_{DAL} employs Eq. 1.

For both datasets the selection mechanism of the active learning process is identical, yet, it can be seen that when using the weak supervision method of the battleship approach the obtained results are slightly better than when relying only the model's confidence. For Amazon-Google, the battleship approaches reaches an AUC score of 467.49, outperforming *battleship* with WS_{DAL} (451.49). Similarly, for Walmart-Amazon the *battleship* approaches also beats its baseline (503.58 and 482.92, respectively). Based on the results, it can be observed that both approaches demonstrate competitive performance. However, the subtle variations in scores emphasize the advantage of the battleship approach's weak supervision method.

6.5 Analysis Conclusion

The parameters α and β are central components of our approach. Anyhow, there is no single setting that is consistently better. A noteworthy observation from our experiments is that incorporating a combination of components (centrality vs. uncertainty and local vs. spatial certainty) yields better performance compared to disregarding a component entirely (*i.e.*, $\alpha \in \{0, 1\}$, $\beta \in \{0, 1\}$). This highlights the challenge of determining optimal values for both alpha and beta in advance, as their interplay and combined effects are crucial for achieving favorable results. Further investigation into the intricate relationship between alpha and beta values may provide valuable insights for future research in this area.

7 RELATED WORK

Active learning refers to an iterative process of selecting data samples to be labeled, which are then used to train a model. The underlying hypothesis is that with a careful selection of informative samples, a classifier can perform as well or even better than when being trained on a fully labeled dataset [53]. Broad range of active learning approaches to the entity matching task have been proposed over the last two decades [48, 51, 61]. By and large, they can be divided into three main groups, namely rule extraction, traditional machine learning, and deep learning.

Qian *et al.* [48] presented a large-scale system of rule learning to improve recall. They focus on identifying atomic matching predicate combinations (operations such as equality and similarity over attributes) that are relevant to identifying matched pairs. Isele and Bizer [18] offer an interactive rule generation method, with the aim of minimizing user involvement in the process. Meduri *et al.* [38]

surveys different sample selection approaches, examining combinations of different approaches alongside combinations of classifiers [51, 61]. A large share of this survey [38] concentrates on the concept of Query-by-committee (QBC) [12, 54] as a policy for sample selection during the active learning process. Typically, QBC finds uncertain samples (which are believed to be more informative to the model) by training multiple versions of a classifier and measuring uncertainty as their level of disagreement. For example, Mozafari *et al.* [40] define the variance of the committee for the matching task as $X(u)(1 - X(u))$ where $X(u)$ is the fraction of classifiers predicted that a given pair is a match.

Deep learning has become the dominant approach to entity matching tasks. Ebraheem *et al.* [24] were the firsts to apply neural networks to entity matching, followed by Mudgal *et al.* [41], which introduced a design space for the use of deep learning to this task. Recently, several work used pre-trained language models to tackle the entity matching problem [7, 29, 30, 45]. In our work, we use DITTO [29] as a major component of the battleship, training it as a matcher after each active learning iteration, and using it to produce tuple pair representations.

Several recent works [5, 19, 25, 63] use deep learning to tackle the entity matching problem under active learning settings. Kasai *et al.* [25] trains a deep learning-based transferable model, aimed at domain prediction task. This model is integrated with active learning process, looking for informative samples to enrich the transferable model in following iterations. Thirumuruganathan *et al.* [63] treat vector-space representations of tuples as features and apply traditional machine learning algorithms to adapt a classifier from one domain to another. Bogatu *et al.* [5] use variational auto-encoders for generating entity representations, then utilize them to create a transferable model. We do not use any designated architecture for domain adaptation, assuming that labeled data from a source domain is not available.

Jain *et al.* [19] use transformer pre-trained language model as a classifier, training conjointly blocker and matcher. For blocking, they obtain a vector-space representation to tuples , and run a similarity search to find potential matches, while for matching they use DITTO [29]. Unlike these aforementioned works, we utilize tuple pair representations (instead of single tuple representation) as part of the sample selection, instead of using it only for prediction [29]. In addition, we expand the notion of uncertainty, which serves as a sample selection criterion [19, 25], allowing spatial considerations to be taken into account. Beyond the scope of the entity matching task, Mahdavi et al. [36] has also utilized sample representations for sample diversity, addressing the error detection task. They based their selection on a feature vector derived from multiple error detection strategies, then employed clustering and label propagation to select representative data samples.

Another approach for dealing with the lack of labeled data was suggested by Wu *et al.* [65]. Their solution, termed ZeroER, does not rely on labeled data at all, but on the assumption that feature vectors of matching pairs (built upon variety of similarity measures) are distributed in a different way than those of non-matching pairs. In our work we use an increasing training set to obtain pair representations (as an alternative to model-agnostic feature vector) and use them to find the most informative samples to label in the following iteration.

8 CONCLUSIONS

In this work we introduce the battleship approach, a novel active learning method for solving the entity matching problem. The approach uses tuple pair representations, utilizing spatial (vector-space) considerations to spot informative data samples that are labeling-worthy. We tailor sample selection to the characteristics of the entity matching problem, establishing the intuition and motivation for the various design choices made in our algorithm. A thorough empirical evaluation shows that the battleship approach is an effective solution to entity matching under low-resource conditions, in some cases even more than a model that was trained against data without labeling

budget limitation. In future work we aspire to expand the battleship approach beyond the entity matching, *e.g.*, to Natural Language Processing tasks, as we believe that some of the main principles that guided us in this work can be generalized and applied to a broader range of challenges.

ACKNOWLEDGMENTS

This work was supported in part by the National Science Foundation (NSF) under award numbers IIS-1956096. We also acknowledge the support of the Benjamin and Florence Free Chair.

REFERENCES

[1] Yoram Baram, Ran El Yaniv, and Kobi Luz. 2004. Online choice of active learning algorithms. *Journal of Machine Learning Research* 5, Mar (2004), 255–291.
[2] Kedar Bellare, Suresh Iyengar, Aditya G Parameswaran, and Vibhor Rastogi. 2012. Active sampling for entity matching. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1131–1139.
[3] Indrajit Bhattacharya and Lise Getoor. 2006. A latent dirichlet model for unsupervised entity resolution. In *Proceedings of the 2006 SIAM International Conference on Data Mining*. SIAM, 47–58.
[4] Mikhail Bilenko and Raymond J Mooney. 2003. Adaptive duplicate detection using learnable string similarity measures. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 39–48.
[5] Alex Bogatu, Norman W Paton, Mark Douthwaite, Stuart Davie, and Andre Freitas. 2021. Cost–effective Variational Active Entity Resolution. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1272–1283.
[6] Paul S Bradley, Kristin P Bennett, and Ayhan Demiriz. 2000. Constrained k-means clustering. *Microsoft Research, Redmond* 20, 0 (2000), 0.
[7] Ursin Brunner and Kurt Stockinger. 2020. Entity matching with transformer architectures-a step forward in data integration. In *International Conference on Extending Database Technology, Copenhagen, 30 March-2 April 2020*. OpenProceedings.
[8] Peter Christen. 2012. The data matching process. In *Data matching*. Springer, 23–35.
[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 4171–4186.
[10] Ivan P Fellegi and Alan B Sunter. 1969. A theory for record linkage. *J. Amer. Statist. Assoc.* 64, 328 (1969), 1183–1210.
[11] Linton C Freeman. 1977. A set of measures of centrality based on betweenness. *Sociometry* (1977), 35–41.
[12] Yoav Freund, H Sebastian Seung, Eli Shamir, and Naftali Tishby. 1997. Selective sampling using the query by committee algorithm. *Machine learning* 28, 2 (1997), 133–168.
[13] Cheng Fu, Xianpei Han, Jiaming He, and Le Sun. 2020. Hierarchical matching network for heterogeneous entity resolution. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*. 3665–3671.
[14] Cheng Fu, Xianpei Han, Le Sun, Bo Chen, Wei Zhang, Suhui Wu, and Hao Kong. 2019. End-to-End Multi-Perspective Matching for Entity Resolution.. In *IJCAI*. 4961–4967.
[15] Lise Getoor and Christopher P Diehl. 2005. Link mining: a survey. *Acm Sigkdd Explorations Newsletter* 7, 2 (2005), 3–12.
[16] Aristides Gionis, Piotr Indyk, Rajeev Motwani, et al. 1999. Similarity search in high dimensions via hashing. In *Vldb*, Vol. 99. 518–529.
[17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *International Conference on Machine Learning*. PMLR, 1321–1330.
[18] Robert Isele and Christian Bizer. 2013. Active learning of expressive linkage rules using genetic programming. *Journal of web semantics* 23 (2013), 2–15.
[19] Arjit Jain, Sunita Sarawagi, and Prithviraj Sen. 2021. Deep Indexed Active Learning for Matching Heterogeneous Entity Representations. *arXiv preprint arXiv:2104.03986* (2021).
[20] Matthew A Jaro. 1989. Advances in record-linkage methodology as applied to matching the 1985 census of Tampa, Florida. *J. Amer. Statist. Assoc.* 84, 406 (1989), 414–420.
[21] Matthew A Jaro. 1995. Probabilistic linkage of large public health data files. *Statistics in medicine* 14, 5-7 (1995), 491–498.
[22] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics* 9 (2021), 962–977.
[23] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[24] Muhammad Ebraheem Saravanan Thirumuruganathan Shafiq Joty and Mourad Ouzzani Nan Tang. 2018. Distributed Representations of Tuples for Entity Resolution. *Proceedings of the VLDB Endowment* 11, 11 (2018).

[25] Jungo Kasai, Kun Qian, Sairam Gurajada, Yunyao Li, and Lucian Popa. 2019. Low-resource Deep Entity Resolution with Transfer and Active Learning. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. 5851–5861.

[26] Pradap Konda et al. 2016. Magellan: Toward building entity matching management systems. *Proceedings of the VLDB Endowment* 9, 12 (2016), 1197–1208.

[27] Vladimir I Levenshtein. 1966. Binary codes capable of correcting deletions, insertions, and reversals. In *Soviet physics doklady*, Vol. 10. 707–710.

[28] Bing Li, Wei Wang, Yifang Sun, Linhan Zhang, Muhammad Asif Ali, and Yi Wang. 2020. GraphER: Token-Centric Entity Resolution with Graph Convolutional Neural Networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 8172–8179.

[29] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. 2020. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment* 14, 1 (2020), 50–60.

[30] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, Jin Wang, Wataru Hirota, and Wang-Chiew Tan. 2021. Deep entity matching: Challenges and opportunities. *Journal of Data and Information Quality (JDIQ)* 13, 1 (2021), 1–17.

[31] Dekang Lin et al. 1998. An information-theoretic definition of similarity. In *ICML*, Vol. 98. 296–304.

[32] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019).

[33] Mingsheng Long, Yue Cao, Jianmin Wang, and Michael Jordan. 2015. Learning transferable features with deep adaptation networks. In *International conference on machine learning*. PMLR, 97–105.

[34] Mingsheng Long, Han Zhu, Jianmin Wang, and Michael I Jordan. 2016. Unsupervised domain adaptation with residual transfer networks. *Advances in neural information processing systems* 29 (2016).

[35] Ilya Loshchilov and Frank Hutter. 2018. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*.

[36] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data*. 865–882.

[37] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[38] Venkata Vamsikrishna Meduri, Lucian Popa, Prithviraj Sen, and Mohamed Sarwat. 2020. A comprehensive benchmark framework for active learning methods in entity matching. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1133–1147.

[39] Zhengjie Miao, Yuliang Li, and Xiaolan Wang. 2021. Rotom: A Meta-Learned Data Augmentation Framework for Entity Matching, Data Cleaning, Text Classification, and Beyond. In *Proceedings of the 2021 International Conference on Management of Data*. 1303–1316.

[40] Barzan Mozafari, Purna Sarkar, Michael Franklin, Michael Jordan, and Samuel Madden. 2014. Scaling up crowd-sourcing to very large datasets: a case for active learning. *Proceedings of the VLDB Endowment* 8, 2 (2014), 125–136.

[41] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data*. 19–34.

[42] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web.* Technical Report. Stanford InfoLab.

[43] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. 2020. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys (CSUR)* 53, 2 (2020), 1–42.

[44] George Papadakis, Jonathan Svirsky, Avigdor Gal, and Themis Palpanas. 2016. Comparative analysis of approximate blocking techniques for entity resolution. *Proceedings of the VLDB Endowment* 9, 9 (2016), 684–695.

[45] Ralph Peeters and Christian Bizer. 2021. Dual-objective fine-tuning of BERT for entity matching. *Proceedings of the VLDB Endowment* 14, 10 (2021), 1913–1921.

[46] Anna Primpeli, Ralph Peeters, and Christian Bizer. 2019. The WDC training dataset and gold standard for large-scale product matching. In *Companion Proceedings of The 2019 World Wide Web Conference*. 381–386.

[47] Michael Prince. 2004. Does active learning work? A review of the research. *Journal of engineering education* 93, 3 (2004), 223–231.

[48] Kun Qian, Lucian Popa, and Prithviraj Sen. 2017. Active learning for large-scale entity resolution. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1379–1388.

[49] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B Gupta, Xiaojiang Chen, and Xin Wang. 2021. A survey of deep active learning. *ACM computing surveys (CSUR)* 54, 9 (2021), 1–40.
[50] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108* (2019).
[51] Sunita Sarawagi and Anuradha Bhamidipaty. 2002. Interactive deduplication using active learning. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 269–278.
[52] Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. 2011. Finding a" kneedle" in a haystack: Detecting knee points in system behavior. In *2011 31st international conference on distributed computing systems workshops*. IEEE, 166–171.
[53] Burr Settles. 2009. Active learning literature survey. (2009).
[54] H Sebastian Seung, Manfred Opper, and Haim Sompolinsky. 1992. Query by committee. In *Proceedings of the fifth annual workshop on Computational learning theory*. 287–294.
[55] Roee Shraga. 2022. HumanAL: Calibrating Human Matching beyond a Single Task. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics* (Philadelphia, Pennsylvania) *(HILDA '22)*. Association for Computing Machinery, New York, NY, USA, Article 7, 8 pages. https://doi.org/10.1145/3546930.3547496
[56] Roee Shraga, Ofra Amir, and Avigdor Gal. 2021. Learning to Characterize Matching Experts. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1236–1247.
[57] Roee Shraga, Gil Katz, Yael Badian, Nitay Calderon, and Avigdor Gal. 2021. From Limited Annotated Raw Material Data to Quality Production Data: A Case Study in the Milk Industry. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 4114–4124.
[58] Rohit Singh, Venkata Vamsikrishna Meduri, Ahmed Elmagarmid, Samuel Madden, Paolo Papotti, Jorge-Arnulfo Quiané-Ruiz, Armando Solar-Lezama, and Nan Tang. 2017. Synthesizing entity matching rules by examples. *Proceedings of the VLDB Endowment* 11, 2 (2017), 189–202.
[59] Parag Singla and Pedro Domingos. 2006. Entity resolution with markov logic. In *Sixth International Conference on Data Mining (ICDM'06)*. IEEE, 572–582.
[60] Chuanqi Tan, Fuchun Sun, Tao Kong, Wenchang Zhang, Chao Yang, and Chunfang Liu. 2018. A survey on deep transfer learning. In *International conference on artificial neural networks*. Springer, 270–279.
[61] Sheila Tejada, Craig A Knoblock, and Steven Minton. 2001. Learning object identification rules for information integration. *Information Systems* 26, 8 (2001), 607–633.
[62] Tippaya Thinsungnoena, Nuntawut Kaoungkub, Pongsakorn Durongdumronchaib, Kittisak Kerdprasopb, and Nittaya Kerdprasopb. 2015. The clustering validity with silhouette and sum of squared errors. *learning* 3, 7 (2015).
[63] Saravanan Thirumuruganathan, Shameem A Puthiya Parambath, Mourad Ouzzani, Nan Tang, and Shafiq Joty. 2018. Reuse and adaptation for entity resolution through transfer learning. *arXiv preprint arXiv:1809.11084* (2018).
[64] Laurens Van der Maaten and Geoffrey Hinton. 2008. Visualizing data using t-SNE. *Journal of machine learning research* 9, 11 (2008).
[65] Renzhi Wu, Sanya Chaba, Saurabh Sawlani, Xu Chu, and Saravanan Thirumuruganathan. 2020. Zeroer: Entity resolution using zero labeled examples. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1149–1164.
[66] Dongxiang Zhang, Dongsheng Li, Long Guo, and Kian-Lee Tan. 2020. Unsupervised entity resolution with blocking and graph algorithms. *IEEE Transactions on Knowledge and Data Engineering* (2020).
[67] Wentao Zhang, Yu Shen, Yang Li, Lei Chen, Zhi Yang, and Bin Cui. 2021. ALG: Fast and Accurate Active Learning Framework for Graph Convolutional Networks. In *Proceedings of the 2021 International Conference on Management of Data*. 2366–2374.
[68] Chen Zhao and Yeye He. 2019. Auto-em: End-to-end fuzzy entity-matching using pre-trained deep models and transfer learning. In *The World Wide Web Conference*. 2413–2424.
[69] Jingbo Zhu, Huizhen Wang, Tianshun Yao, and Benjamin K Tsou. 2008. Active learning with sampling by uncertainty and density for word sense disambiguation and text classification. In *Proceedings of the 22nd International Conference on Computational Linguistics (Coling 2008)*. 1137–1144.

Received April 2023; accepted August 2023